Modelos de Calidad de Software

Escuela de Tecnología

Mg. Daniel Arias, **PMP™** y **Scaled Scrum Master™**

¿Qué aprendimos la clase pasada?

- Gestión de la Calidad:
  - Métricas
  - ISO 9126
  - ISO 25000
  - ISO 14598
  - Moprosoft
  - CMMI y SCAMPI
  - ITIL

idat

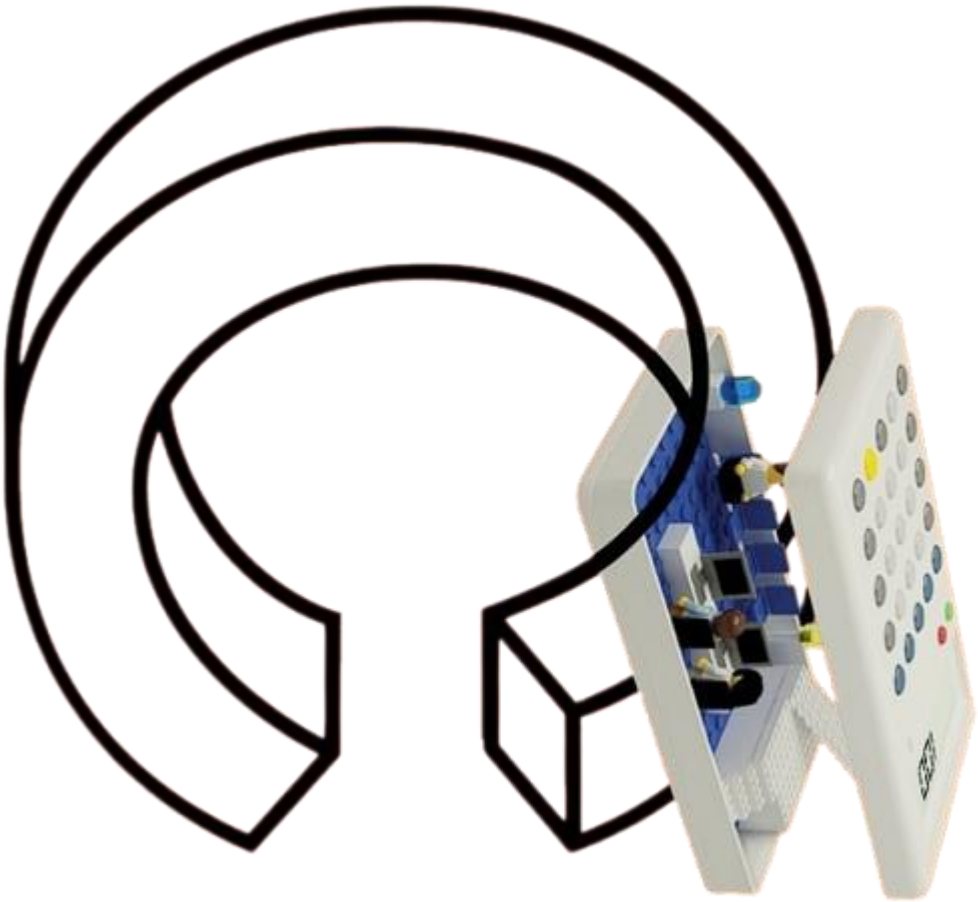
Conversemos

http://www.menti.com

idat

Introducción a Calidad de Software – Parte 3

Semana 3



idat

Logro de aprendizaje

- Definición de Pruebas de Software
- ¿Cuántas pruebas son suficientes?
- Causas de los defectos de software.
- Ciclo de vida de las pruebas de software
- Los 7 principios de las pruebas de software

Actividad de aprendizaje

ACTIVIDAD VIRTUAL

Revisa y analiza el enlace sugerido en la plataforma virtual luego realiza la actividad propuesta

- ¿Qué son las pruebas de software?
- ¿Cuál es la importancia de las pruebas?
- Con sus propias palabras explique los 7 principios de pruebas.
- ¿Cuáles son los entregables por capa etapa de las pruebas?.

idat

Logro de aprendizaje

Diferenciar la calidad y pruebas y software, considerando el ciclo de vida de las pruebas y sus principios.

idat

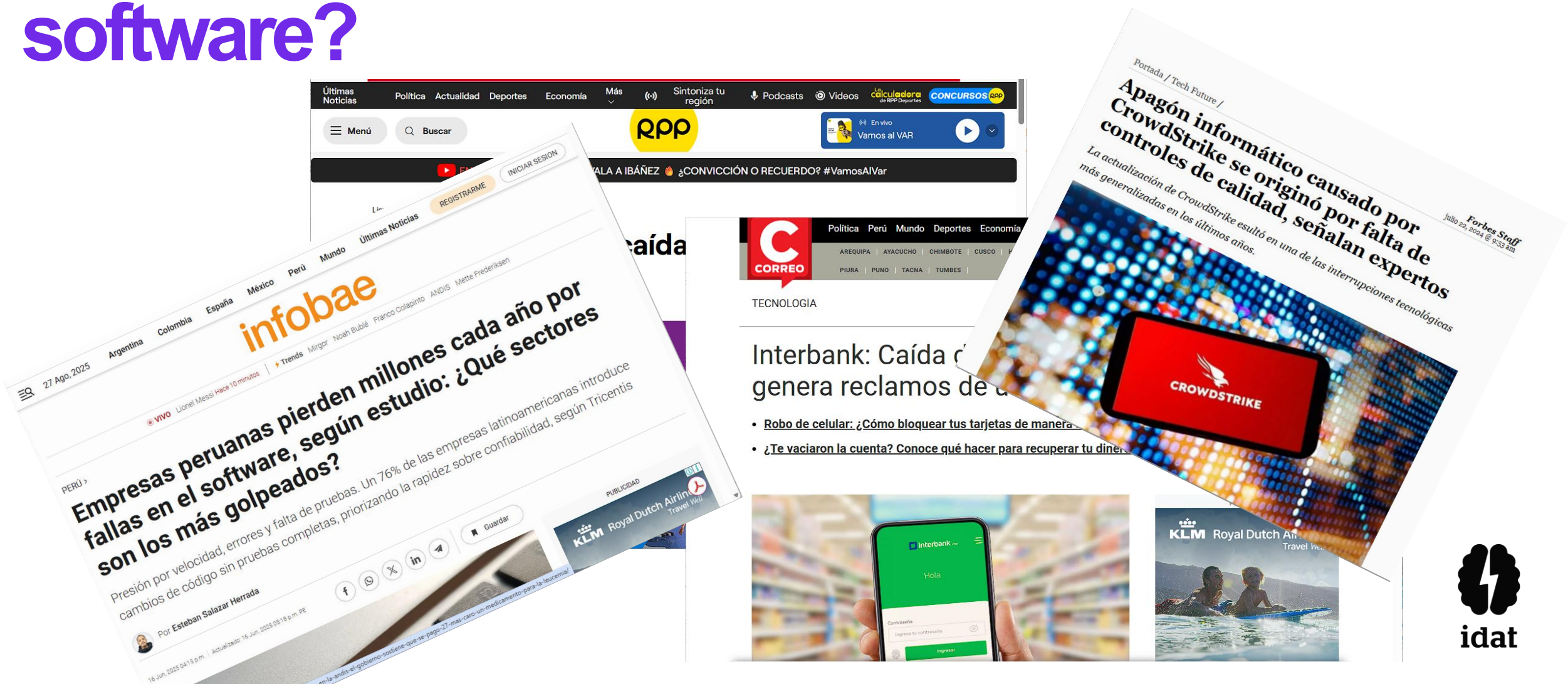
¿Qué son las pruebas de software?

Las pruebas de software son el proceso de evaluar y verificar que un producto o aplicación de software hace lo que se supone que debe hacer. Los beneficios de unas buenas pruebas incluyen la prevención de errores y la mejora del rendimiento.

Las pruebas de software hoy en día son más efectivas cuando son continuas, lo que indica que las pruebas se inician durante el diseño, continúan a medida que se crea el software e incluso se producen cuando se despliega en producción.

idat

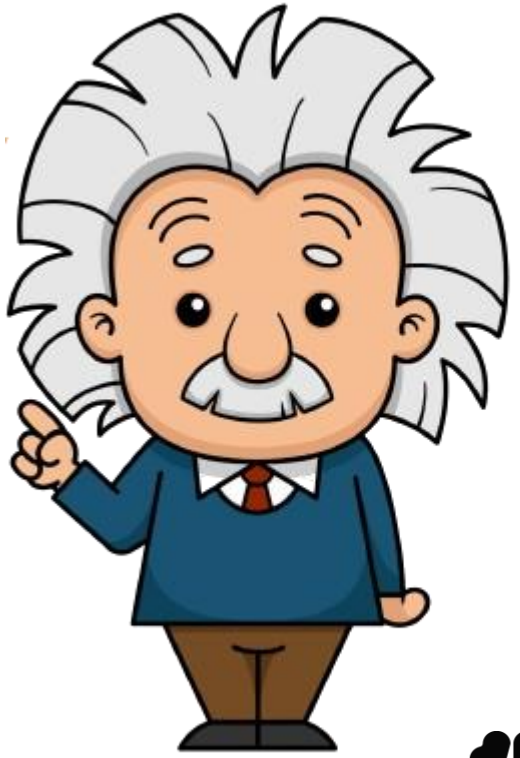
¿Por qué son importantes las pruebas de software?



idat

¿Por qué son importantes las pruebas de software?

Las pruebas de software son la culminación del desarrollo de aplicaciones, mediante las cuales los testers evalúan el código, cuestionándolo. Esta evaluación puede ser breve o continuar hasta que todas las partes interesadas estén satisfechas. Las pruebas de software identifican errores y problemas durante el proceso de desarrollo para que se solucionen antes del lanzamiento del producto. Este enfoque garantiza que solo se distribuyan a los consumidores productos de calidad, lo que a su vez aumenta la satisfacción y la confianza del cliente .



idat

¿Por qué son importantes las pruebas de software?

Las siguientes son razones importantes por las que las técnicas de pruebas de software deben incorporarse al desarrollo de aplicaciones:

- **Identifica defectos con anticipación.** El desarrollo de aplicaciones complejas puede dar lugar a errores. Las pruebas de software son fundamentales, ya que identifican cualquier problema o defecto en el código escrito para que puedan solucionarse antes de la entrega del producto.
- **Mejora la calidad del producto.** En cuanto al atractivo para el cliente, ofrecer un producto de calidad es una métrica importante a considerar. Un producto excepcional solo se puede entregar si se prueba eficazmente antes del lanzamiento. Las pruebas de software ayudan al producto a superar el control de calidad (QA) y a cumplir con los criterios y especificaciones definidos por los usuarios.

idat

¿Por qué son importantes las pruebas de software?

- **Aumenta la confianza y la satisfacción del cliente.** Probar un producto a lo largo de su ciclo de desarrollo genera confianza y satisfacción, ya que permite conocer sus puntos fuertes y débiles. Al recibir el producto, los clientes lo han probado repetidamente y cumple con los estándares de calidad.
- **Detecta vulnerabilidades de seguridad.** El código de aplicación inseguro puede generar vulnerabilidades que los atacantes pueden explotar. Dado que la mayoría de las aplicaciones están en línea hoy en día, pueden ser un medio de ciberataques y deben probarse exhaustivamente durante las distintas etapas de su desarrollo. Una aplicación web publicada sin las pruebas de software adecuadas puede ser fácilmente víctima de un ataque de secuencias de comandos entre sitios (CSE), en el que los atacantes intentan inyectar código malicioso en el navegador web del usuario accediendo a través de la aplicación web vulnerable.

idat

¿Por qué son importantes las pruebas de software?

- **Ayuda con la escalabilidad.** Las pruebas de escalabilidad , un tipo de proceso de prueba de software no funcional, se realizan para evaluar la capacidad de una aplicación para escalar con cargas de trabajo crecientes, como el tráfico de usuarios, el volumen de datos y el número de transacciones. También pueden identificar el punto en el que una aplicación podría dejar de funcionar y las razones, como alcanzar o superar un umbral determinado, como el número total de usuarios simultáneos de la aplicación.
- **Ahorra dinero.** Los problemas de desarrollo de software que pasan desapercibidos debido a la falta de pruebas pueden afectar a las organizaciones posteriormente y generar un mayor coste. Tras el lanzamiento de la aplicación, puede ser más difícil rastrear y resolver los problemas, ya que la aplicación de parches de software suele ser más costosa que las pruebas durante las etapas de desarrollo.

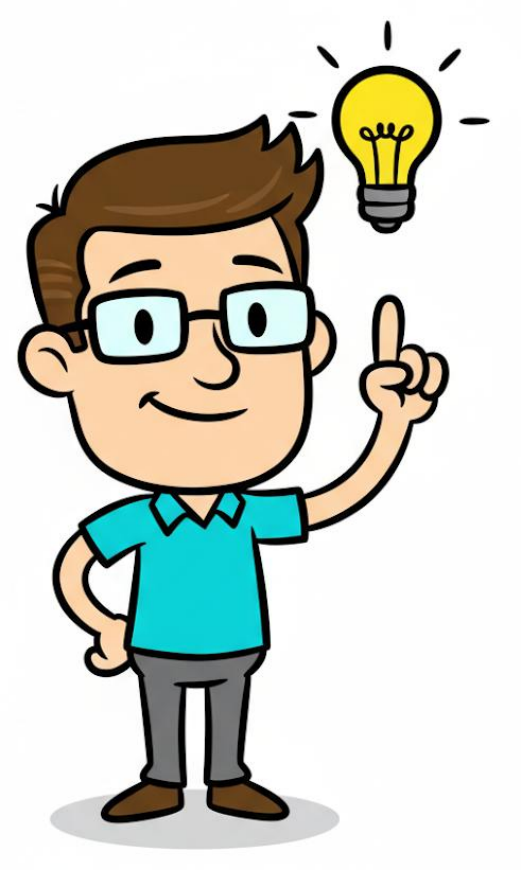
idat

¿Cuántas pruebas son suficientes?

- "Sigue haciendo pruebas hasta conseguir una cobertura del 100%".
- *"Haz pruebas hasta que no encuentres ningún error".*
- *"Pruebe hasta que cada línea de código se haya ejecutado bajo todas las condiciones posibles".*
- *"Las pruebas deben continuar hasta que todas las partes interesadas estén completamente satisfechas con el software".*

idat

¿Cuándo empezar a realizar pruebas?



- ¡Debes comenzar a probar tan pronto como tengas algo para probar!
- En pocas palabras, las pruebas deben comenzar en las primeras etapas del proceso de desarrollo, no solo al final. Esto se enfatiza aún más con enfoques de desarrollo modernos como la metodología ágil y las pruebas Shift-Left[1].
- Las pruebas comienzan y continúan durante todo el ciclo de vida del desarrollo de software .

[1] Las pruebas Shift-Left (desplazamiento a la izquierda) es una práctica de desarrollo de software que consiste en integrar las pruebas y la detección de errores en las etapas más tempranas del ciclo de vida del desarrollo, como el diseño y los requisitos, en lugar de esperar hasta el final. Este enfoque proactivo busca identificar y corregir problemas lo antes posible, lo que reduce costos, acelera el tiempo de entrega y mejora la calidad general del producto.

idat

¿Cuándo empezar a realizar pruebas?

- **Comience las pruebas con anticipación:** Comience incluso cuando partes del proyecto aún se estén desarrollando. Esto se conoce como pruebas de desplazamiento a la izquierda , lo que significa que se adelantan las pruebas en una etapa más temprana del proceso. Esto ayuda a detectar problemas a tiempo, antes de que se agraven.
- **Pruebas a medida que construyes:** A medida que los desarrolladores escriben código o crean funcionalidades, pueden empezar a probar pequeñas partes para asegurarse de que funcionen correctamente. Estas podrían ser pruebas unitarias , que verifican partes individuales del código. Los enfoques más populares incluyen el desarrollo guiado por pruebas (TDD) y el desarrollo impulsado por el negocio (BDD) .

idat

¿Cuándo empezar a realizar pruebas?

- **Probar las funciones a medida que se completan:** Una vez finalizada una función o un módulo, pruébelo para garantizar que cumpla con los requisitos y funcione según lo previsto. En este punto, se realizan pruebas como las de integración o las funcionales.
- **Pruebas continuas:** Si utiliza pruebas automatizadas, deben ejecutarse continuamente a medida que se implementan nuevos cambios. Esto proporciona retroalimentación inmediata al equipo y les ayuda a detectar problemas rápidamente.
- **Pruebas con usuarios reales:** una vez que el sistema esté listo, realice pruebas con usuarios para ver cómo interactúan los usuarios reales con el software y verificar si hay problemas de usabilidad.

idat

¿Cuándo dejar de realizar pruebas?

Sabemos que es recomendable realizar pruebas durante todo el proceso de desarrollo. Sin embargo, es necesario saber cuándo finalizar un ciclo de pruebas. Nos referimos, en esencia, a los criterios de salida de las pruebas.

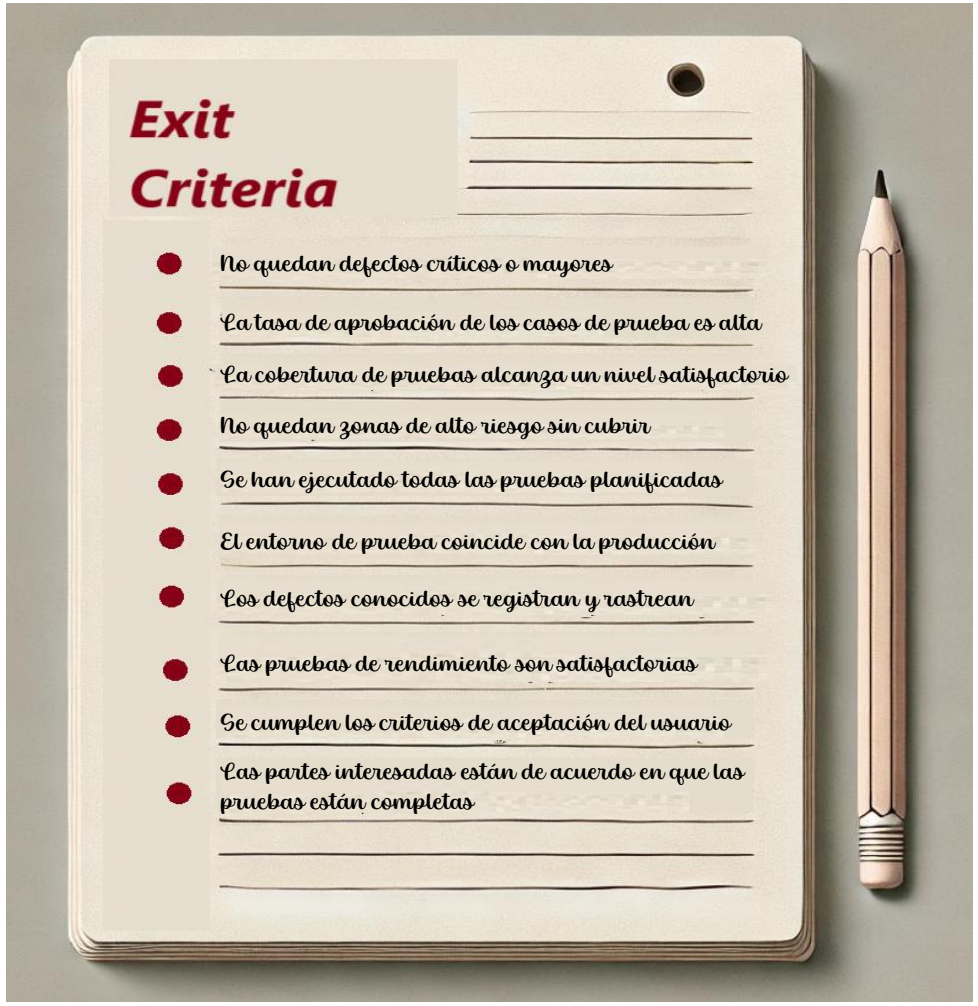
idat

¿Cuándo dejar de realizar pruebas?

- **¿Qué son los criterios de salida de la prueba?**

  Los criterios de salida de prueba son las condiciones que deben cumplirse antes de poder detener las pruebas de una aplicación de software. Considérelo como una lista de verificación para asegurar que el software esté listo para su lanzamiento. Estos criterios ayudan a los equipos a saber cuándo han realizado suficientes pruebas y pueden avanzar a la siguiente fase, como la implementación.

  Esto es lo que normalmente verá involucrado como parte de los criterios de salida.



idat

¿Cuándo dejar de realizar pruebas?

- **Factores que determinan cuándo detener el proceso de prueba**

  Si bien no existe un enfoque único para determinar si se han realizado suficientes pruebas, aquí se presentan algunos parámetros comunes que influyen en esta decisión. Puede usarlos para determinar los criterios de salida de su proyecto.

idat

¿Cuándo dejar de realizar pruebas?

- **Cobertura de la prueba**
  - **Qué significa:** La cobertura de pruebas se refiere a la cantidad de software que se ha probado. Es como asegurarse de haber revisado todas las piezas de un auto para comprobar su funcionamiento antes de conducirlo.
  - **Por qué es importante:** Cuantas más áreas del software pruebes, menos probable será que pases por alto errores. Pero no es necesario probar todas las combinaciones posibles; céntrate en las partes más importantes.
  - **Cómo ayuda:** Si ha probado todas las características y funciones clave que los usuarios usarán con mayor frecuencia, puede estar más seguro de que el software está listo. Puede que no sea necesario probarlo todo.

idat

¿Cuándo dejar de realizar pruebas?

- **Tasa de defectos**
  - **Qué significa:** La tasa de defectos es la cantidad de errores o problemas detectados durante las pruebas. Si se detectan errores constantemente, podría indicar que se necesitan más pruebas. Si los errores son poco frecuentes y los detectados son menores, podría ser el momento de detenerse.
  - **Por qué es importante:** Si las pruebas revelan menos defectos con el tiempo, esto indica que el software se está volviendo más estable. Si se siguen detectando defectos con regularidad, indica que el software no está listo para su lanzamiento.
  - **Cómo ayuda:** Monitorear la tasa de defectos le ayuda a determinar cuándo se han resuelto la mayoría de los problemas y puede detener las pruebas. Una tasa de defectos alta significa que necesita más pruebas, pero una tasa baja indica que son suficientes.

idat

¿Cuándo dejar de realizar pruebas?

- **Pruebas basadas en riesgos**
  - **Qué significa:** Las pruebas basadas en riesgos implican centrarse en las partes más críticas o de mayor riesgo del software. Por ejemplo, si su software gestiona pagos, conviene probar esa área con mayor profundidad, ya que un error en esa función podría ser muy costoso.
  - **Por qué es importante:** Algunas partes del software son más importantes que otras. Priorizar las pruebas en estas áreas garantiza que se pruebe lo más importante.
  - **Cómo ayuda:** Al identificar y probar las características de alto riesgo de manera más rigurosa, puede dejar de probar una vez que esté seguro de que las partes principales de alto riesgo son estables y seguras.

idat

¿Cuándo dejar de realizar pruebas?

- **Pruebas de regresión**
  - **Qué significa:** La prueba de regresión consiste en verificar si los nuevos cambios o características han malogrado algo que solía funcionar.
  - **Por qué es importante:** Tras realizar cambios en el software, es fundamental asegurarse de que las funciones que antes funcionaban no se vean afectadas. Si todas las funciones críticas siguen funcionando después de los cambios, significa que las pruebas podrían ser suficientes.
  - **Cómo ayuda:** Si las pruebas de regresión muestran que no hay nada importante malogrado, puede detener las pruebas y estar seguro de que los cambios no introdujeron nuevos problemas.

idat

¿Cuándo dejar de realizar pruebas?

- **Fechas límite de lanzamiento**
  - **Qué significa:** A veces, es necesario detener las pruebas porque hay una fecha límite fija para lanzar el software. Esto podría deberse a necesidades del negocio, presiones del mercado o eventos planificados.
  - **Por qué es importante:** Aunque las pruebas no estén completamente completas, en algún momento el software debe publicarse. Esto implica encontrar un equilibrio entre la necesidad de realizar pruebas exhaustivas y el cumplimiento de los plazos.
  - **Cómo ayuda:** Si se acerca la fecha límite y se han realizado suficientes pruebas en las áreas más importantes, es posible que deba detenerlas. La clave está en gestionar las expectativas y centrarse en los problemas de alta prioridad.

idat

¿Cuándo dejar de realizar pruebas?

- **Consistencia de los resultados de las pruebas**
  - **Qué significa:** Si obtiene resultados consistentes y repetibles en sus pruebas, especialmente si la mayoría de las pruebas pasan, entonces es una buena señal de que el software es estable.
  - **Por qué es importante:** Los resultados consistentes demuestran que el software no presenta nuevos problemas con cada ronda de pruebas. Si se encuentran menos defectos de forma constante, es señal de que las pruebas posteriores podrían no aportar mucha información nueva.
  - **Cómo ayuda:** Una vez que los resultados de las pruebas se estabilicen y los defectos que encuentre sean menores o no críticos, podría ser el momento de detener las pruebas y lanzar el producto.

idat

¿Cuándo dejar de realizar pruebas?

- **Automatización en pruebas**
  - **Qué significa:** La automatización se refiere al uso de herramientas y scripts para ejecutar pruebas repetitivas, lo que acelera el proceso y brinda una respuesta más rápida.
  - **Por qué es importante:** Las pruebas automatizadas pueden indicarle rápidamente si los nuevos cambios causan problemas en el software, por lo que no tiene que hacer todo manualmente.
  - **Cómo ayuda:** Si las pruebas automatizadas se ejecutan con regularidad y funcionan correctamente, esto puede ayudarle a determinar cuándo se han realizado suficientes pruebas. Las pruebas automatizadas proporcionan información rápida, por lo que puede detener las pruebas una vez que confirmen la estabilidad.

idat

¿Cuándo dejar de realizar pruebas?

- **Aprobación de los Stakeholders**
  - **Qué significa:** Los Stakeholders son las personas con un interés particular en el software, como propietarios de empresas, gerentes de proyecto o clientes. Estas personas deben estar de acuerdo en que el software está listo para su lanzamiento.
  - **Por qué es importante:** Las pruebas no se limitan a detectar errores, sino también a satisfacer las necesidades de la empresa y de los usuarios. Los Stakeholders pueden aportar información valiosa sobre si el software está listo para su lanzamiento.
  - **Cómo ayuda:** Una vez que las partes interesadas estén satisfechas con los resultados de las pruebas y se sientan seguras de que el software cumple con los requisitos comerciales, puede detener las pruebas.

idat

¿Cuándo dejar de realizar pruebas?

- **Recursos de prueba**
  - **Qué significa:** Las pruebas requieren tiempo, personal y herramientas. A veces, es posible que deba detenerlas porque ha alcanzado el límite de recursos disponibles, ya sea tiempo, personal o presupuesto.
  - **Por qué es importante:** Las pruebas no son ilimitadas. Debe priorizar dónde enfocar los esfuerzos de prueba. Si ya ha probado las partes más críticas y ha agotado los recursos, quizás sea momento de detenerse.
  - **Cómo ayuda:** Cuando te quedas sin recursos pero has cubierto las áreas más importantes, es una buena señal de que has realizado suficientes pruebas.

¿Cuándo dejar de realizar pruebas?

- **Rendimiento y escalabilidad**
  - **Qué significa:** Las pruebas de rendimiento garantizan que el software funcione sin problemas incluso con un uso intensivo, y las pruebas de escalabilidad garantizan que pueda manejar más usuarios o datos en el futuro.
  - **Por qué es importante:** Si el software puede manejar la carga esperada y escalar según sea necesario, es menos probable que surjan problemas después del lanzamiento.
  - **Cómo ayuda:** Una vez que se aprueban las pruebas de rendimiento y escalabilidad , y no hay problemas importantes, puede ayudar a decidir si las pruebas son suficientes.

idat

¿Cuándo deben detenerse las pruebas? El papel de la automatización

Con las pruebas automatizadas, puedes acelerar el proceso de prueba. Esto te ayuda a decidir cuándo es suficiente.

- **Retroalimentación más rápida:** Cuando las pruebas se ejecutan rápidamente, se pueden identificar problemas con antelación. Si las pruebas automatizadas funcionan correctamente de forma consistente, es una clara señal de que el software es estable y es posible que no sea necesario realizar más pruebas.
- **Consistencia en las pruebas:** La consistencia significa que los resultados de las pruebas son fiables. No se obtendrán falsos positivos ni falsos negativos debido a errores humanos. Si las pruebas automatizadas se ejecutan de forma consistente en diferentes ciclos, esto indica que hay menos errores y que el software probablemente esté listo para su lanzamiento.

idat

¿Cuándo deben detenerse las pruebas? El papel de la automatización

- **Pruebas de regresión más fáciles:** cuando las pruebas de regresión siguen pasando después de cada actualización, sugiere que el software es estable y es seguro detener las pruebas sin pasar por alto problemas críticos.
- **Mayor cobertura de pruebas:** si la automatización ha cubierto un alto porcentaje de su base de código o funciones, y todas las pruebas están pasando, puede dejar de realizar pruebas porque ya ha cubierto las áreas críticas.
- **Ideal para pruebas repetitivas:** Algunas pruebas deben ejecutarse repetidamente, como las de rendimiento o las de la misma función en diferentes entornos. La automatización es ideal para ejecutar estas pruebas una y otra vez sin intervención humana.

idat

¿Cuándo deben detenerse las pruebas? El papel de la automatización

- **Monitoreo y alertas más sencillos:** La automatización puede monitorear activamente el sistema para detectar problemas, de modo que los evaluadores no tengan que verificar manualmente cada resultado. Con el monitoreo automatizado, puede tener la seguridad de que, si surge algún problema, se detectará de inmediato. Si no se detecta ningún problema, podría ser el momento de detener las pruebas.
- **Eficiencia y rentabilidad:** La automatización hace que las pruebas sean más eficientes y rentables. Una vez escritas, las pruebas automatizadas pueden ejecutarse tantas veces como sea necesario sin coste adicional. También pueden programarse para ejecutarse fuera del horario laboral.
- **Pruebas continuas:** Es más fácil integrar conjuntos de pruebas automatizadas en los pipelines de CI/CD y ejecutarlos cuando lo desee. De esta forma, puede probar cada compilación o versión rápidamente y tener confianza en el estado de su aplicación.

idat

Pruebas eficientes con herramientas de automatización de pruebas basadas en IA

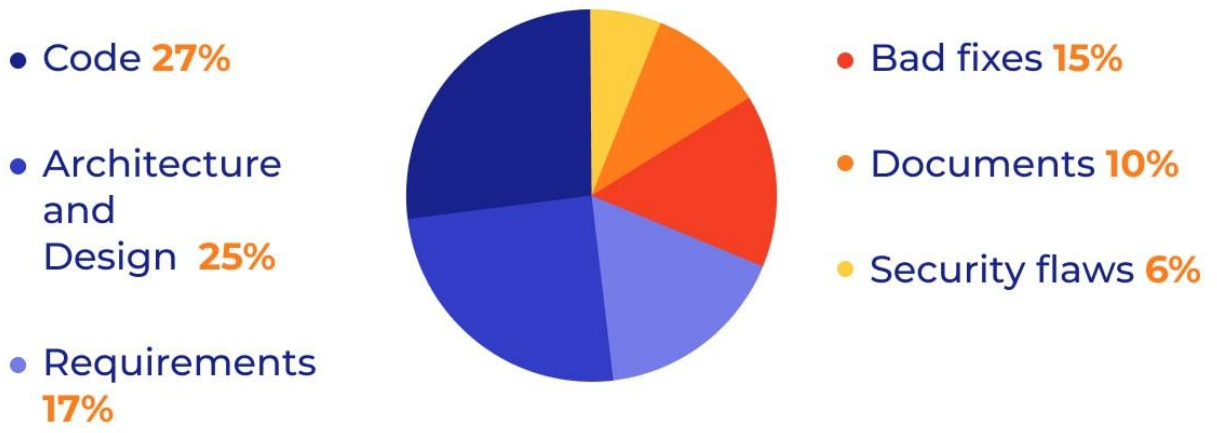
Si bien la automatización de pruebas puede ser la clave para saber cuándo has realizado suficientes pruebas, elegir una herramienta deficiente puede arruinar tus esfuerzos. Opta por una herramienta fácil de usar, que te permita escribir casos de prueba fácilmente, requiera un mantenimiento mínimo y te ofrezca ejecuciones de prueba fiables. Una de las mejores opciones para lograr este tipo de automatización de pruebas sin esfuerzo es testRigor. Si integras testRigor con tu framework de CI/CD, tendrás un sistema que puede realizar pruebas con la misma frecuencia y rapidez que tus lanzamientos.

Esta herramienta generativa basada en IA garantiza que pueda ejecutar y mantener fácilmente sus casos de prueba. testRigor es un agente de IA reconocido por la fiabilidad de sus ejecuciones de prueba, por lo que puede estar seguro de que su aplicación se está probando exhaustivamente. Esto le dará la confianza necesaria para tomar las medidas necesarias y decidir cuándo finalizar las pruebas.

idat

Causas de los defectos de software

Las causas de los defectos de software incluyen errores humanos como la ambigüedad en los requisitos, la codificación incorrecta y la falta de habilidades, problemas en el diseño y la arquitectura, fallas en las pruebas adecuadas y en el control de calidad, y factores externos como la entrada de datos inesperada, la interacción de componentes y los cambios en el entorno de ejecución.

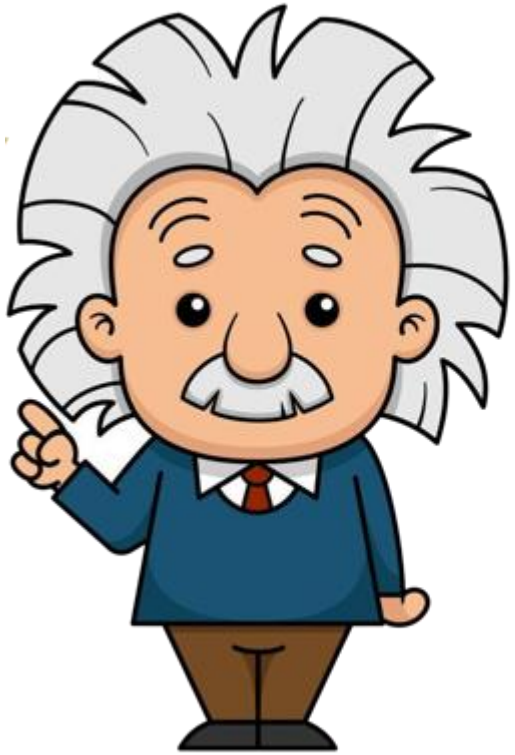


Source: Capers Jones, Quantifying Software, 2018 CRC Press

idat

Causas de los defectos de software

- **Causas relacionadas con el proceso de desarrollo**
  - **Errores de codificación y diseño:** Los desarrolladores pueden introducir errores humanos al escribir código o tomar decisiones de diseño incorrectas.
  - **Requisitos ambiguos o incompletos:** La falta de claridad o la constante modificación de los requisitos puede generar confusión y errores en el software resultante.
  - **Pruebas insuficientes:** No realizar pruebas exhaustivas puede dejar pasar defectos que se presentarían en el uso real del software.



idat

Causas de los defectos de software

- **Causas relacionadas con el software y su entorno**
  - **Interacciones inesperadas:** Los componentes del software pueden interactuar de forma no prevista, generando comportamientos anómalos.
  - **Entrada de datos inesperada:** El software puede fallar al procesar datos que no se encuentran en el formato esperado o que contienen valores incorrectos.
  - **Condiciones de borde no manejadas:** Situaciones extremas o casos límite que no fueron considerados durante el diseño pueden provocar errores.
  - **Dependencia de recursos externos:** Fallos o comportamientos inesperados de bases de datos o servicios web pueden afectar la operación del software.
  - **Cambios en el entorno de ejecución:** Actualizaciones del sistema operativo, de hardware o de otras aplicaciones pueden crear incompatibilidades y generar defectos.

idat

Ciclo de vida de las pruebas de software

Es un conjunto de fases o etapas estructuradas que describen el proceso completo de testing, desde la planificación inicial hasta el cierre de las actividades de prueba.

No es un modelo independiente, sino que se integra dentro del Ciclo de Vida de Desarrollo de Software (SDLC) para garantizar que la calidad se evalúe de manera sistemática y no solo al final del desarrollo.

Objetivo Principal

El objetivo principal del ciclo de vida de pruebas es establecer un proceso organizado y eficiente para encontrar defectos, evaluar la calidad del producto y proporcionar información objetiva sobre su estado, minimizando riesgos y asegurando que se cumplan los requisitos.

¿Por Qué es Crucial un Ciclo de Pruebas Definido?

Estructura Clara
Proporciona un plan organizado y sistemático para todas las pruebas.

Eficiencia Óptima
Optimiza la asignación de recursos y el tiempo, evitando duplicidades.

Calidad Garantizada
Asegura una cobertura completa, identificando defectos a tiempo.

Trazabilidad Completa
Vincula cada prueba con un requisito, facilitando la auditoría.

idat

Las Fases Clave: Análisis y Planificación

El ciclo de vida se compone de actividades planificadas y ejecutadas de manera secuencial o solapada, comenzando con una sólida base.

1. Análisis de Requisitos

   Se analizan a fondo los requisitos del software y la documentación para entender el alcance.

   - Revisión de especificaciones.
   - Identificación de áreas críticas.

2. Planificación de Pruebas

   Se define la estrategia general, recursos, tiempos y entornos necesarios.

   - Definición del enfoque de testing.
   - Estimación de esfuerzos y riesgos.

Resultado: Un **Plan de Pruebas** detallado.

idat

Las Fases Clave: Diseño y Especificación

Diseño y Especificación de Pruebas

En esta fase, convertimos los requisitos en casos de prueba concretos y preparamos todo el material necesario.

- Creación de **Casos de Prueba** detallados derivados de los requisitos.
- Diseño de **Datos de Prueba** realistas y representativos.
- Preparación del **Entorno de Testing** (hardware, software, configuraciones).

Resultado: Casos de Prueba listos, Datos de Prueba, y Ambiente de testing configurado.

idat

Las Fases Clave: Ejecución y Gestión de Defectos

La ejecución es el corazón del ciclo, donde los casos de prueba se materializan y los defectos salen a la luz.

01

Ejecución de Casos

Se ejecutan los casos de prueba planificados.

02

Registro de Resultados

Éxito o fallo de cada caso, con evidencias.

03

Reporte de Defectos

Los errores se documentan detalladamente en herramientas de gestión.

04

Seguimiento

Gestión del ciclo de vida del defecto hasta su cierre.

Resultado: Registros de ejecución, reportes de defectos y métricas de avance.

idat

Las Fases Clave: Cierre y Evaluación

Cierre y Evaluación de Pruebas

Esta fase consolida los aprendizajes y valida la calidad final del producto.

- Evaluación de los resultados frente a los **criterios de salida**.
- Creación de **Informes Finales** y métricas clave (ej. defectos resueltos vs. pendientes).
- Archivo de los activos de prueba para **reutilización futura** (ej. pruebas de regresión).

Resultado: Reporte Final de Pruebas y documento de Lecciones Aprendidas.

idat

Los 7 Principios Fundamentales del Testing de Software

¿Por qué son importantes estos principios?

Estos principios son la base de cualquier estrategia de testing efectiva. Nos ayudan a:

- Guiar las actividades de testing.
- Utilizar los recursos de forma eficiente.
- Establecer expectativas realistas.
- Mejorar la comunicación con los stakeholders.
- Fundamentar las decisiones sobre la calidad del software.

idat

Los 7 Principios Fundamentales del Testing de Software

Principio 1: Las pruebas muestran la presencia de defectos

El testing es como un detective: puede encontrar pistas de que hay problemas (defectos), pero nunca puede emitir un veredicto.

- Nunca podemos probar que un software está **"completamente" libre de defectos**.
- El testing reduce significativamente la probabilidad de que queden defectos sin descubrir.
- Nuestro enfoque principal es la **identificación** de defectos críticos e importantes.

idat

Los 7 Principios Fundamentales del Testing de Software

Principio 2: El testing exhaustivo es imposible

En la mayoría de los casos, probar todas las combinaciones posibles de entradas y estados de un software es una tarea inabarcable. Es una realidad que debemos aceptar.

1

Testing basado en riesgos

Priorizar las áreas más críticas o propensas a fallos.

2

Diseño de pruebas

Aplicar técnicas inteligentes para seleccionar los casos de prueba más representativos.

3

Análisis de coste/beneficio

Evaluar el valor del testing frente al esfuerzo requerido.

idat

Los 7 Principios Fundamentales del Testing de Software

Principio 3: Pruebas tempranas

"Cuanto antes, mejor" es el mantra aquí. **Integrar el testing desde el inicio** del ciclo de desarrollo previene problemas mayores y reduce costes.

Actividades tempranas clave:

- Revisión exhaustiva de requisitos y especificaciones.
- Análisis estático de código para detectar errores antes de la ejecución.
- Pruebas de concepto y prototipos.

idat

Los 7 Principios Fundamentales del Testing de Software

Principio 4: Agrupamiento de defectos

También conocido como la Ley de Pareto aplicada al testing, este principio observa que la mayoría de los defectos se concentran en unos pocos módulos o componentes del software.

Enfoque en áreas problemáticas
Dirigir el testing intensivo donde más se necesita, basándonos en datos históricos y análisis de riesgos.

Análisis de causas raíz
Identificar por qué ciertos módulos son más defectuosos para mejorar los procesos de desarrollo.

Uso eficiente de recursos
Optimizar el esfuerzo de testing, maximizando la detección de defectos con menos recursos.

Este enfoque permite una mejora continua del proceso.

idat

Los 7 Principios Fundamentales del Testing de Software

Principio 5: Paradoja del pesticida

Si se ejecutan las mismas pruebas repetidamente, con el tiempo dejarán de encontrar nuevos defectos. Es como si el software se "hiciera inmune" a ellas.v

Actualizar casos de prueba
Revisar y modificar las pruebas existentes para cubrir nuevas funcionalidades o áreas problemáticas.

Diversificar técnicas
Incorporar diferentes tipos de testing: exploratorio, de regresión, de rendimiento, etc.

Pruebas exploratorias
Permiten descubrir defectos inesperados mediante la experimentación y el aprendizaje.

idat

Los 7 Principios Fundamentales del Testing de Software

Principio 6: El testing depende del contexto

No existe un enfoque "talla única" para el testing. Cada proyecto tiene sus particularidades que dictan la estrategia más adecuada.

- **Tipo de aplicación:** Una web, una app móvil o un sistema embebido no se prueban igual.
- **Regulaciones:** Software en sectores regulados (salud, finanzas) requiere un testing más riguroso.
- **Riesgos del negocio:** Priorizar lo que más impacto tenga en la empresa si falla.
- **Restricciones:** Tiempo y presupuesto siempre influirán en el alcance y la profundidad del testing.

idat

Los 7 Principios Fundamentales del Testing de Software

Principio 7: La falacia de ausencia de errores

Un software sin errores técnicos no es necesariamente un software útil. Podemos construir algo "correctamente", pero si no cumple las necesidades del usuario, habrá fallado.

> **"¿Estamos construyendo el producto correcto y lo estamos construyendo correctamente?"**

- El testing debe validar el **valor de negocio**.
- La satisfacción del usuario final es tan crucial como la funcionalidad.

idat

Integración de los principios: Hoja de ruta para el testing

Aplicar estos 7 principios de forma conjunta maximiza la eficacia del testing y el valor entregado.

01

Comenzar temprano

Identificar y prevenir defectos desde las fases iniciales.

02

Enfocarse en el riesgo

Priorizar las áreas críticas y de mayor impacto.

03

Diversificar las pruebas

Evitar la paradoja del pesticida con nuevas técnicas.

04

Adaptar al contexto

Personalizar la estrategia según las necesidades del proyecto.

05

Buscar defectos

Reconocer que no todos serán encontrados.

06

Validar el valor

Asegurar que el software no solo funciona, sino que satisface las necesidades del usuario.

idat

Conclusiones

- El testing no es solo "ejecutar pruebas", es un proceso que comienza desde el primer momento.
- Cada fase tiene entradas, actividades y resultados específicos.
- La retroalimentación a los desarrolladores es un output crucial de la fase de ejecución.
- Un ciclo bien ejecutado es la base para entregar software con mayor calidad y confianza.

idat

¿Dudas?

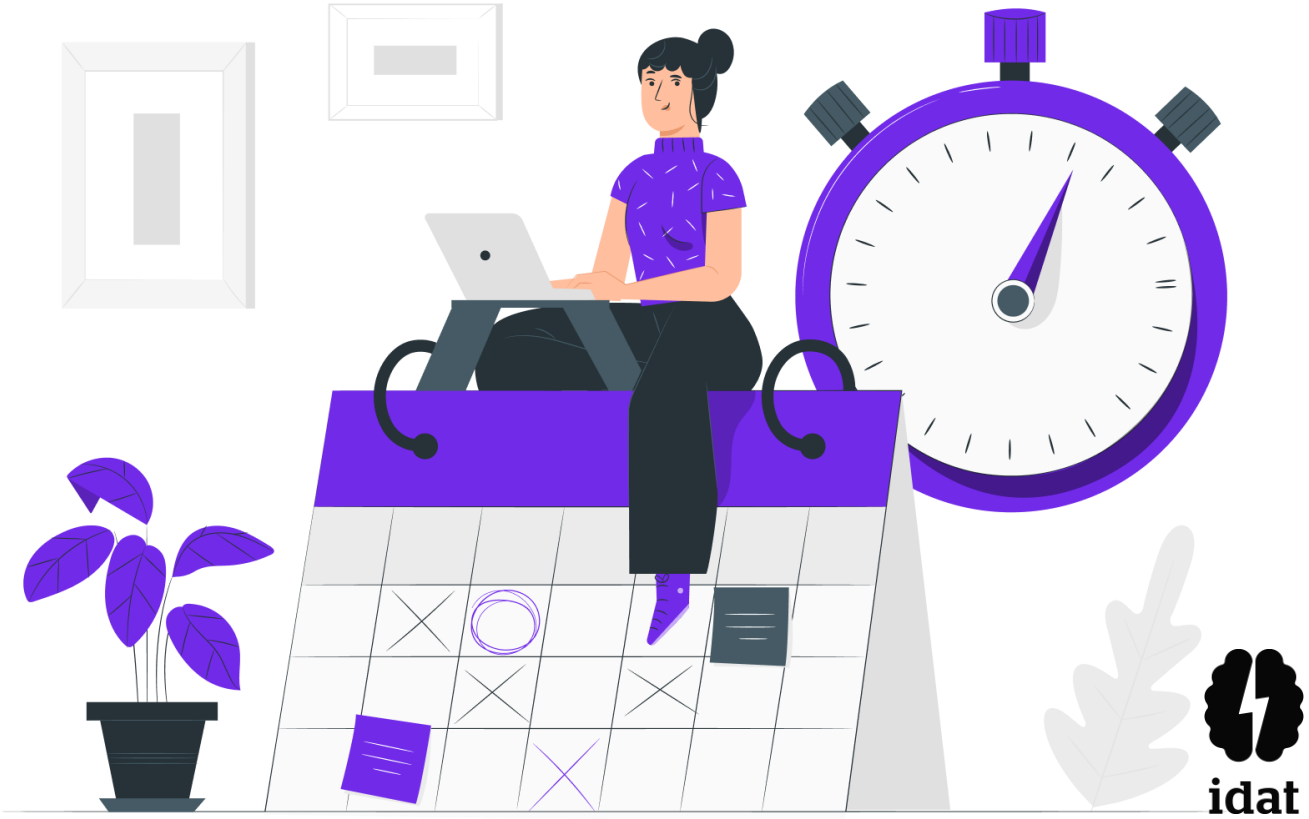
¿Miedos?

¿Temores?

idat

Aplicamos lo aprendido: Asignación para la clase siguiente

Resolver la actividad en el EVA



idat

idat

GRACIAS